Object-Oriented Programming and Data Structures

COMP2012: Pointer, Reference, New C++11 Features, C++ Class Revision & const-ness

Cecia Chan
Brian Mak
Pedro Sander

Department of Computer Science & Engineering
The Hong Kong University of Science and Technology
Hong Kong SAR, China

Why Take This Course?

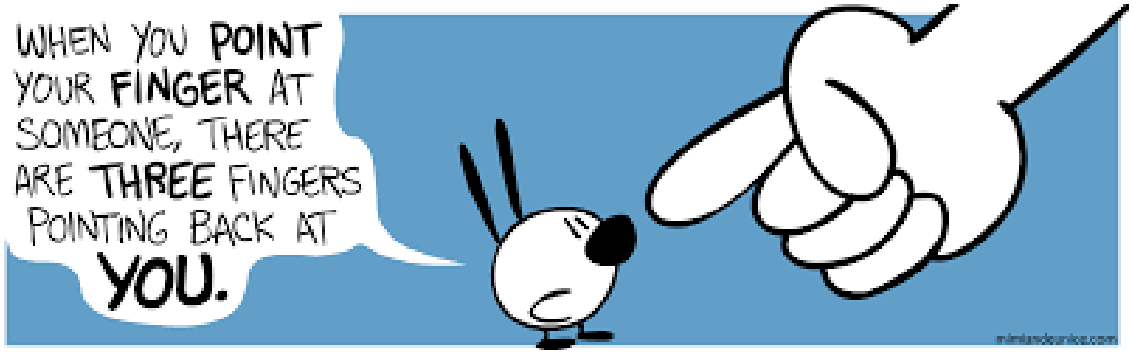
You have taken COMP1021/1022P and COMP2011. So you can program already, right?

- Think about this: You have been learning English for many years, but can you write a novel?
- You basically have learned the C part of C++ in COMP2011 with a brief introduction to C++ classes, and you can write small C++ programs.
- But what if you are going to write a large program, probably with a team of programmers?

In this course, you will learn the essence of OOP with some new C++ constructs with an aim to write large softwares.

Part I

Quick Review: Reference and Pointer



Variable, Reference Variable, Pointer Variable

```
1   #include <iostream>     /* File: confusion.cpp */
2   using namespace std;
3
4   int x = 5;              // An int variable
5   int& xref = x;          // A reference variable: xref is an alias of x
6   int* xptr = &x;         // A pointer variable: xptr points to x
7
8   void xprint()
9   {
10      cout << hex << endl; // Print numbers in hexadecimal format
11      cout << "x =     " << x     << "\t\tx address     = " << &x << endl;
12      cout << "xref =  " << xref  << "\t\txref address  = " << &xref << endl;
13      cout << "xptr =  " << xptr  << "\txptr address  = "   << &xptr << endl;
14      cout << "*xptr = " << *xptr << endl;
15  }
16
17  int main()
18  {
19      x += 1; xprint();
20      xref += 1; xprint();
21      xptr = &xref; xprint(); // Now xptr points to xref
22
23      return 0;
24  }
```

Pointer vs. Reference

Reference can be thought as a special kind of pointer, but there are 3 big differences:

1. A pointer can point to nothing (`nullptr`), but a reference is always bound to an object.
2. A pointer can point to different objects at different times (through assignments). A reference is always bound to the same object.

   Assignments to a reference does not change the object it refers to but only the value of the referenced object.
3. The name of a pointer refers to the pointer itself. The `*` or `->` operators have to be used to access the underlying object it points to.

   The name of a reference always refers to the object. There are no special operators for references.

Part II

Some New Features in C++11

A List of New Features in C++11

- uniform and general initialization using { }-list ⋆
- type deduction of variables from initializer: auto
  — NOT ALLOWED TO USE IN COMP2011/2012
- prevention of narrowing ⋆
- generalized and guaranteed constant expressions: constexpr
- Range-for-statement ⋆
- null pointer keyword: nullptr ⋆
- scoped and strongly typed enums: enum_class
- rvalue references, enabling move semantics †
- lambdas or lambda expressions ⋆
- support for unicode characters
- long long integer type
- delegating constructors †
- in-class member initializers †
- explicit conversion operators †
- override control keywords: override and final †

General Initialization Using { }-Lists

- In the past, you always initialize variables using the assignment operator $=$.

Example: $=$ Initializer

```
int x = 5;
float y = 9.8;
int& xref = x;
int a[] = {1, 2, 3};
```

- C++11 allows the more uniform and general curly-brace-delimited initializer list.

Example: { } Initializer

```
int x = {5};        // = here is optional
float y {9.8};
int& xref {x};
int a[] {1, 2, 3};
```

Initializer Example 1

```
1   #include <iostream>     /* File: initializer1.cpp */
2   using namespace std;
3
4   int main()
5   {
6       int w = 3.4;
7       int x1 {6};
8       int x2 = {8};       // = here is optional
9       const int y {'k'};
10      int z {6.4};        // Error!
11
12      cout << "w = " << w << endl;
13      cout << "x1 = " << x1 << endl << "x2 = " << x2 << endl;
14      cout << "y = " << y << endl << "z = " << z << endl;
15
16      int& ww = w;
17      int& www {ww}; www = 123;
18      cout << "www = " << www << endl;
19      return 0;
20  }
```

```
initializer1.cpp:10:15: error: narrowing conversion of 6.4000000000000004e+0
from double to int inside { } [-Wnarrowing]
     int z {6.4};
               ^
```

Initializer Example 2

```
1   #include <iostream>     /* File: initializer2.cpp */
2   using namespace std;
3
4   int main()
5   {
6       const char s1[] = "Steve Jobs";
7       const char s2[] {"Bill Gates"};
8       const char s3[] = {'h', 'k', 'u', 's', 't', '\0'};
9       const char s4[] {'h', 'k', 'u', 's', 't', '\0'};
10
11      cout << "s1 = " << s1 << endl;
12      cout << "s2 = " << s2 << endl;
13      cout << "s3 = " << s3 << endl;
14      cout << "s4 = " << s4 << endl;
15      return 0;
16  }
```

Differences Between the = and { } Initializers

- The { } initializer is more restrictive: it doesn't allow conversions that lose information — narrowing conversions.
- The { } initializer is more general as it also works for:
  - arrays
  - other aggregate structures
  - class objects (we'll talk about that later)

Range-for Statements

- In the past, you write a for-loop by
  - initializing an index variable,
  - giving an ending condition, and
  - writing some post-processing that involves the index variable.

Example: Traditional for-Loop

```
for (int k = 0; k < 5; ++k)
    cout << k*k << endl;
```

- C++11 adds a more flexible range-for syntax that allows looping through a sequence of values specified by a list.

Example: Range-for-Loops

```
for (int k : { 0, 1, 2, 3, 4 })
    cout << k*k << endl;

for (int k : { 1, 19, 54 }) // Numbers need not be successive
    cout << k*k << endl;
```

Range-for Example

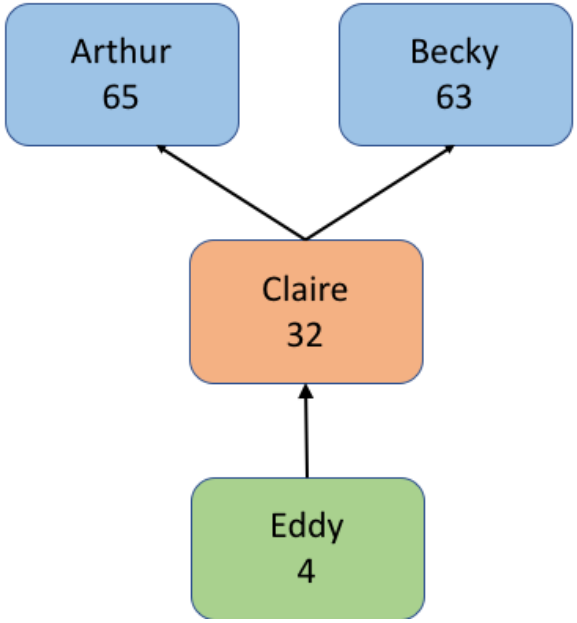
```
1   #include <iostream>     /* File : range-for.cpp */
2   using namespace std;
3
4   int main()
5   {
6       cout << "Square some numbers in a list" << endl;
7       for (int k : {0, 1, 2, 3, 4})
8           cout << k*k << endl;
9
10      int range[] { 2, 5, 27, 40 };
11
12      cout << "Square the numbers in range" << endl;
13      for (int k : range)  // Won't change the numbers in range
14          cout << k*k << endl;
15
16      cout << "Print the numbers in range" << endl;
17      for (int v : range) cout << v << endl;
18
19      for (int& x : range) // Double the numbers in range in situ
20          x *= 2;
21
22      cout << "Again print the numbers in range" << endl;
23      for (int v : range) cout << v << endl;
24      return 0;
25  }
```

Part III

A Revision Example: Person and Family

A Revision Example: Person & Family

- It consists of the class Person, from which families are built.
- A person, in general, has at most 1 child, and his/her father and mother may or may not be known.
- The information of his/her family includes him/her and his/her parents and grandparents from both of his/her parents.



Revision Example: Expected Output

```
Name: Arthur
Father: unknown
Mother: unknown
Grand Fathers: unknown, unknown
Grand Mothers: unknown, unknown

Name: Becky
Father: unknown
Mother: unknown
Grand Fathers: unknown, unknown
Grand Mothers: unknown, unknown

Name: Claire
Father: Arthur
Mother: Becky
Grand Fathers: unknown, unknown
Grand Mothers: unknown, unknown

Name: Eddy
Father: unknown
Mother: Claire
Grand Fathers: unknown, Arthur
Grand Mothers: unknown, Becky
```

Revision Example: Person Class — Header File

```
1   #include <iostream>     /* File: person.h */
2   using namespace std;
3
4   class Person
5   {
6     private:
7       char* _name;
8       int _age;
9       Person *_father, *_mother, *_child;
10
11    public:
12      Person(const char* my_name, int my_age, Person* my_father = nullptr,
13             Person* my_mother = nullptr, Person* my_child = nullptr);
14      ~Person();
15
16      Person* father() const;
17      Person* mother() const;
18      Person* child() const;
19      void print_age() const;
20      void print_name() const;
21      void print_family() const;
22
23      void have_child(Person* baby) ;
24  };
```

Revision Example: Person Class — Implementation File

```
1   #include "person.h"     /* File: person.cpp */
2   #include <cstring>
3
4   Person::Person(const char* my_name, int my_age, Person* my_father,
5                  Person* my_mother, Person* my_child)
6   {
7       _name = new char [strlen(my_name)+1];
8       strcpy(_name, my_name);
9       _age = my_age;
10      _father = my_father;
11      _mother = my_mother;
12      _child = my_child;
13  };
14
15  Person::~Person() { delete [] _name; }
16
17  Person* Person::father() const { return _father; }
18
19  Person* Person::mother() const { return _mother; }
20
21  Person* Person::child() const { return _child; }
22
23  void Person::have_child(Person* baby) { _child = baby; }
```

Revision Example: Person Class — Implementation File ..

```
24
25  void Person::print_age() const { cout << _age; }
26
27  void Person::print_name() const
28  {
29      cout << (_name ? _name : "unknown");
30  }
31
32
33
34
35  // Helper function
36  void print_parent(Person* parent)
37  {
38      if (parent)
39          parent->print_name();
40      else
41          cout << "unknown";
42  }
43
44
45
46
47
```

Revision Example: Person Class — Implementation File ..

```
48  void Person::print_family() const
49  {
50      Person *f_grandfather = nullptr, *f_grandmother = nullptr,
51          *m_grandfather = nullptr, *m_grandmother = nullptr;
52
53      if (_father) {
54          f_grandmother = _father->mother();
55          f_grandfather = _father->father();
56      }
57
58      if (_mother) {
59          m_grandmother = _mother->mother();
60          m_grandfather = _mother->father();
61      }
62
63      cout << "Name: "; print_name(); cout << endl;
64      cout << "Father: "; print_parent(_father); cout << endl;
65      cout << "Mother: "; print_parent(_mother); cout << endl;
66
67      cout << "Grand Fathers: "; print_parent(f_grandfather);
68      cout << ", "; print_parent(m_grandfather); cout << endl;
69      cout << "Grand Mothers: "; print_parent(f_grandmother);
70      cout << ", "; print_parent(m_grandmother); cout << endl;
71  }
```

Revision Example: Family Building Test Program

```
1   #include "person.h"     /* File: family.cpp */
2
3   int main()
4   {
5       Person arthur("Arthur", 65, nullptr, nullptr, nullptr);
6       Person becky("Becky", 63, nullptr, nullptr, nullptr);
7       Person claire("Claire", 32, &arthur, &becky, nullptr);
8       Person eddy("Eddy", 4, nullptr, &claire, nullptr);
9
10      arthur.have_child(&claire);
11      becky.have_child(&claire);
12      claire.have_child(&eddy);
13
14      arthur.print_family(); cout << endl;
15      becky.print_family();  cout << endl;
16      claire.print_family(); cout << endl;
17      eddy.print_family();   cout << endl;
18      return 0;
19  }
```

Part IV

General Remarks on C++ Classes

Structure vs. Class

In C++ , structures are special classes and they may have member functions. By default,

$$\text{struct } \{ \ldots \} \equiv \text{class } \{ \text{ public: } \ldots \}$$
$$\text{class } \{ \ldots \} \equiv \text{struct } \{ \text{ private: } \ldots \}$$

```
#include <iostream>     /* File: struct/person.h */
using namespace std;
struct Person
{
    char* _name;
    int _age;
    Person *_father, *_mother, *_child;
    Person(const char* my_name, int my_age, Person* my_father = nullptr,
           Person* my_mother = nullptr, Person* my_child = nullptr);
    ~Person();
    Person* father() const;
    Person* mother() const;
    Person* child() const;
    void print_age() const;
    void print_name() const;
    void print_family() const;
    void have_child(Person* baby) ;
};
```

Class Name: Name Equivalence

- A class definition introduces a new abstract data type.
- C++ relies on name equivalence (and not structure equivalence) for class types.

```
class X { int a; };
class Y { int a; };
class W { int a; };
class W { int b; };  // Error, double definition

X x;
Y y;

x = y;  // Error: type mismatch
```

Class Data Members

Data members can be any basic type, or any user-defined types if they are already declared.

Below are special cases:

- A class name can be used inside its own definition for a pointer to an object of the class:

```
class Cell
{
    int info;
    Cell* next;
};
```

Class Data Members ..

- A forward declaration of a class X can be used in the definition of another class Y to define a pointer to X:

```
class Cell;         // Forward declaration of Cell

class List
{
    int size;
    Cell* data;     // Points to a (forward-declared) Cell object
    Cell x;         // Error: Cell not defined yet!
};

class Cell          // Definition of Cell
{
    int info;
    Cell* next;
};
```

Default Initializer for Non-static Members (C++11)

```
class Complex
{
  private:
    float real = 1.3;   // Note: not allowed before C++11
    float imag {0.5};   // Use either = or { } initializer
  public:
    ...
};
```

- You are advised to initialize non-static data member values by
  - class constructors
  - class member initializer list in a constructor
  - class member functions
- Non-static data members that are not initialized by the 3 ways above will have the values of their default member initializers if they exist, otherwise their values are undefined.
- We'll talk about static vs. non-static members later. All data members you'll see most of the time are non-static.

Class Member Functions

- These are the functions declared inside the body of a class.
- They can be defined in 3 ways:

1. as inline functions within the class body. The keyword inline is optional in this case.

```
class Person
{   ...
    Person* child() const { return _child; }
    void have_child(Person* baby) { _child = baby; }
};
```

Or,

```
class Person
{   ...
    inline Person* child() const { return _child; }
    inline void have_child(Person* baby) { _child = baby; }
};
```

Class Member Functions ..

2. as inline functions, but outside the class body, in the same header file. In this case, the keyword inline is mandatory. It also requires the additional prefix consisting of the class name and the class scope operator ::
   ⇒ to enhance readability especially when the class body consists of a few lines of code.

```
1  /* File: person.h */
2  class Person
3  {   ...
4      inline Person* child() const;
5      inline void have_child(Person* baby);
6  };
7
8  inline Person* Person::child() const { return _child; }
9  inline void Person::have_child(Person* baby) { _child = baby; }
```

Class Member Functions ...

3. as non-inline functions, outside the class body, in a separate implementation .cpp file. Then add the prefix consisting of the class name and the class scope operator ::
   ⇐ any benefits of doing this?

```
/* File: person.h */
class Person
{   ...
    Person* child() const;
    void have_child(Person* baby);
};

/* File: person.cpp */
Person* Person::child() const { return _child; }
void Person::have_child(Person* baby) { _child = baby; }
```

Class Scope and Scope Operator ::

- C++ uses lexical (static) scope rules: the binding of name occurrences to declarations are done statically at compile-time.
- Identifiers declared inside a class definition are under its scope.
- To define the members functions outside the class definition, prefix the identifier with the class scope operator ::
- e.g., `temperature::kelvin()`, `temperature::celsius()`

```
int height = 10;
class Weird
{
    short height;
    Weird() { height = 5; }
};
```

Q1 : Which "height" is used in `Weird::Weird()`?

Q2 : Can we access the global height inside the Weird class body?

this Pointer

- Each class member function implicitly contains a pointer of its class type named "this".
- When an object calls the function, this pointer is set to point to the object.
- For example, after compilation, the member function `Person::have_child(Person* baby)` of `Person` will be translated to a unique global function by adding a new argument:

```
void Person::have_child(Person* this, Person* baby)
{
    this->_child = baby;
}
```

- The call, `becky.have_child(&eddy)` becomes

  `Person::have_child(&becky, &eddy)`.

Example: Return an Object by this — complex.h

```
1   class Complex            /* File: complex.h */
2   {
3     private:
4       float real; float imag;
5
6     public:
7       Complex(float r, float i) { real = r; imag = i; }
8       void print() const { cout << "(" << real << " , " << imag << ")" << endl; }
9
10      Complex add1(const Complex& x)  // Return by value
11      {
12          real += x.real; imag += x.imag;
13          return (*this);
14      }
15      Complex* add2(const Complex& x) // Return by value using pointer
16      {
17          real += x.real; imag += x.imag;
18          return this;
19      }
20      Complex& add3(const Complex& x) // Return by reference
21      {
22          real += x.real; imag += x.imag;
23          return (*this);
24      }
25  };
```

Example: Return an Object by this — complex-test.cpp

```cpp
#include <iostream>     /* File: complex-test.cpp */
using namespace std;
#include "complex.h"

void f(const Complex a) { a.print(); }   // const Complex a  = u
void g(const Complex* a) { a->print(); } // const Complex* a = &u
void h(const Complex& a) { a.print(); }  // const Complex& a = u

int main()
{
    // Check the parameter passing methods
    Complex u(4, 5); f(u); g(&u); h(u);

    // Check the parameter returning methods
    Complex w(10, 10); cout << endl << endl;
    Complex x(4, 5); (x.add1(w)).print();    // Complex  temp = *this = x
    Complex y(4, 5); (y.add2(w))->print();   // Complex* temp =  this = &y
    Complex z(4, 5); (z.add3(w)).print();    // Complex& temp = *this = z

    cout << endl << endl;           // What is the output now?
    Complex a(4, 5); a.add1(w).add1(w).print();   a.print(); cout << endl;
    Complex b(4, 5); b.add2(w)->add2(w)->print(); b.print(); cout << endl;
    Complex c(4, 5); c.add3(w).add3(w).print();   c.print();
    return 0;
}
```

Return-by-Value and Return-by-Reference

There are 2 ways to pass parameters to a function

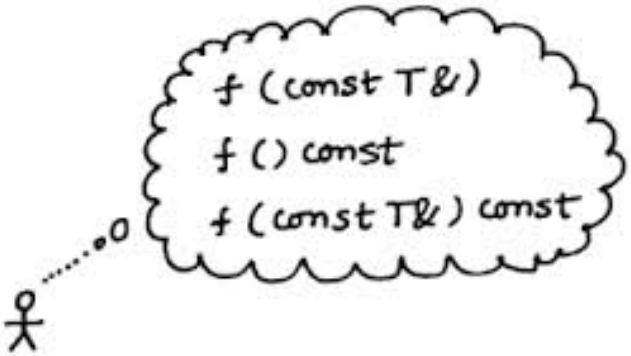
- pass-by-value (PBV)
- pass-by-reference (PBR)
  - lvalue reference: that is what you learned in the past and we'll keep just saying *reference* for lvalue reference.
  - rvalue reference (C++11)

Similarly, you may return from a function by returning an object's

- value: the function will make a separate copy of the object and return it. Changes made to the copy have no effect on the original object.
- (lvalue) reference: the object itself is passed back! Any further operations on the returned object will directly modify the original object as it is the same as the returned object.
- rvalue reference: we'll talk about this later.

Part V

const-ness



const

- const, in its simplest usage, is used to express a user-defined constant — a value that can't be changed.

  ```
  const float PI = 3.1416;
  ```

- Some people like to write const identifiers in capital letters.
- In the old days, constants are defined by the #define preprocessor directive:

  ```
  #define PI 3.1416
  ```

  Question: Any shortcomings?

- const actually may be used to represent more than just numerical constants, but also const objects, pointers, and even member functions!
- The const keyword can be regarded as a safety net for programmers: If an object should not change, make it const.

Example: Constant Object of User-defined Types

```
/* File: const-object-date.h */

class Date      // There are problems with this code; what are they?
{
  private:
    int year, month, day;

  public:
    Date() { cin >> year >> month >> day; }
    Date(int y, int m, int d) { year = y; month = m; day = d; }

    void add_month() { month += 1; }; // Will be an inline function

    int difference(const Date& d)
    { /* Incomplete: write this function */ }

    void print()
        { cout << year << "/" << month << "/" << day << endl; }
};
```

Example: Constant Object of User-defined Types ..

```
1  #include <iostream>     /* File: const-object-date.cpp */
2  using namespace std;
3  #include "const-object-date.h"
4
5  int main()   // There are problems with this code; what are they?
6  {
7      const Date WW2(1945, 9, 2); // World War II ending date
8      Date today;
9      WW2.print();
10     today.print();
11
12     // How long has it been since World War II?
13     cout << "Today is " << today.difference(WW2)
14          << " days after WW2" << endl;
15
16     // What about next month?
17     WW2.add_month();    // Error; do you mean today.add_month()??
18     cout << today.difference(WW2) << " days by next month.\n";
19
20     return 0;
21 }
```

const Member Functions

- To indicate that a class member function does not modify the class object — its data member(s), one can (and should!) place the const keyword after the argument list.

```
class Date                    /* File: const-object-date2.h */
{
  private:
    int year, month, day;

  public:
    Date() { cin >> year >> month >> day; }
    Date(int y, int m, int d) { year = y; month = m; day = d; }

    void add_month() { month += 1; }; // Will be an inline function

    int difference(const Date& d) const { /* Incomplete */ }
    void print() const
        { cout << year << "/" << month << "/" << day << endl; }
};
```

const Member Functions and this Pointer

- A const object can only call const member functions of its class.
- But a non-const object can call both const and non-const member functions of its class.
- The this pointer in const member functions points to const objects. For example,

  ▷ `int Date::difference(const Date& d) const;` is compiled to

  ```
  int Date::difference(const Date* this, const Date& d);
  ```

  ▷ `void Date::print() const;` is compiled to

  ```
  void Date::print(const Date* this);
  ```

- Thus, the object calling const member function becomes const inside the function and cannot be modified.

const and const Pointers

- When a pointer is used, two objects are involved:
  - the pointer itself
  - the object being pointed to
- The syntax for pointers to constant objects and constant pointers can be confusing. The rule is that
  - any const to the left of the $*$ in a declaration refers to the object being pointed to.
  - any const to the right of the $*$ refers to the pointer itself.
- It can be helpful to read these declarations from right to left.

```
/* File: const-char-ptrs1.cpp */
char c = 'Y';
char *const cpc = &c;
char const* pcc;
const char* pcc2;
const char *const cpcc = &c;
char const *const cpcc2 = &c;
```

Example: const and const Pointers

```
1  #include <iostream>     /* File: const-char-ptrs2.cpp */
2  using namespace std;
3
4  int main()
5  {
6      char s[] = "COMP2012"; // Usual initialization in the past
7      char p[] {"MATH1013"}; // C++11 style of uniform initialization
8
9      const char* pcc {s};   // Pointer to constant char
10     pcc[5] = '5';          // Error!
11     pcc = p;               // OK, but what does that mean?
12
13     char *const cpc = s;   // Constant pointer
14     cpc[5] = '5';          // OK
15     cpc = p;               // Error!
16
17     const char *const cpcc = s; // const pointer to const char
18     cpcc[5] = '5';         // Error!
19     cpcc = p;              // Error!
20     return 0;
21 }
```

const and const Pointers ..

Having a pointer-to-const pointing to a non-const object doesn't make that object a constant!

```
/* File: const-int-ptr.cpp */
int i = 151;
i += 20;    // OK

int* pi = &i;
*pi += 20;  // OK

const int* pic = &i;
*pic += 20; // Error! Can't change i through pic

pic = pi;   // OK
*pic += 20; // Error! Can't change *pi thru pic

pi = pic;   // Error: Invalid conversion from 'const int*' to 'int*'
```

const References as Function Arguments

- There are 2 good reasons to pass an argument as a reference. What are they?
- You can (and should!) express your intention to leave a reference argument of your function unchanged by making it const.
- There are 2 advantages:

1. If you accidentally try to modify the argument in your function, the compiler will catch the error.

```
void cbr(int& x) { x += 10; }        // Fine

void cbcr(const int& x) { x += 10; } // Error!
```

const References as Function Arguments ..

2. You may pass both const and non-const arguments to a function that requires a const reference parameter.

   Conversely, you may pass only non-const arguments to a function that requires a non-const reference parameter.

```
#include <iostream>
using namespace std;
void cbr(int& a) { cout << a << endl; }
void cbcr(const int& a) { cout << a << endl; }
int main()
{
    int x {50}; const int y {100};
    // Which of the following give(s) compilation error?
    cbr(x);
    cbcr(x);
    cbr(y);
    cbcr(y);
    cbr(1234);
    cbcr(1234);
}
```

Summary: Good Practice

- Objects you don't intend to change ⇒ const objects

```
const double PI = 3.1415927;
const Date handover(1, 7, 1997);
```

- Function arguments you don't intend to change
  ⇒ const arguments

```
void print_height(const Large_Obj& LO){ cout << LO.height(): }
```

- Class member functions that don't change the data members
  ⇒ const member functions

```
int Date::get_day() const { return day; }
```

Summary

- Regarding which objects can call const or non-const member functions:

Calling Object	const Member Function	non-const Member Function
const Object	√	X
non-const Object	√	√

- Regarding which objects can be passed to functions with const or non-const reference/pointer arguments:

Passing Object	const Function Argument	non-const Function Argument
literal constant	√	X
const Object	√	X
non-const Object	√	√